

Algoritmi e Strutture Dati–parte I

lez. 17

Francesco Camastra

francesco.camastra@uniparthenope.it

Docente

- Prof. Francesco Camastra

Orario lezioni

- Mercoledì 11.00-13.00
(aula 11, Secondo Piano)
- Giovedì 9.00-11.00 (aula 11, Secondo Piano)

Ricevimento

- Orario: Mercoledì 14.00-18.00
- Dove: Stanza 430, Quarto piano, Centro Direzionale, Isola C4
- Per il ricevimento è **OBBLIGATORIO** inviare una email di prenotazione almeno due giorni prima del giorno di ricevimento.
- email del Docente:
francesco.camastra@uniparthenope.it
- Pagina del Docente:
dist.uniparthenope.it/francesco.camastra

Prerequisiti

- Programmazione I
- Matematica I
- Programmazione II
- **NOTA BENE:** Le propedeuticità non sono obbligatorie ... Ossia potreste anche sostenere l'esame senza aver dato Programmazione I e Programmazione II

Frequenza

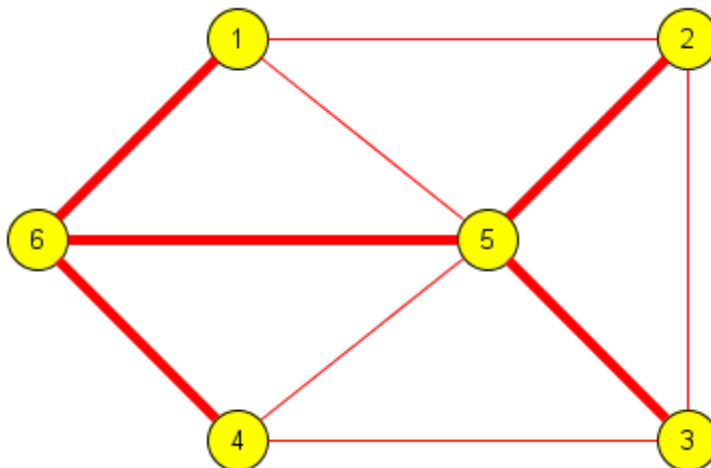
- La Frequenza del corso non è obbligatoria

Modalità dell' esame

- Prova scritta di Algoritmi e Strutture Dati I ovvero Due prove intercorso (validità quattro appelli)
- Svolgimento di un progetto Algoritmi e Strutture Dati II da scegliersi tra una lista di progetti che verrà comunicata dal docente (validità quattro appelli).
- Superato lo scritto di ASD-I ed il progetto di ASD-II si accede all' Esame Orale congiunto di ASD-I e ASD-II

Albero ricoprente

- Dato un grafo connesso e non orientato G , un albero ricoprente (**spanning tree**) di G è un subgrafo che è un albero e connette tutti i vertici.



Albero ricoprente minimo

- Un grafo può avere differenti **spanning trees**.
- Se assegniamo un peso (*weight*) a ciascun arco, possiamo utilizzarlo per assegnare un peso allo spanning tree sommando i pesi degli archi nello spanning tree. Un **albero ricoprente minimo** o **minimum spanning tree (MST)** è lo spanning tree che ha il peso minimo.
- Un'applicazione del MST è la progettazione dei circuiti elettronici dove si vuole minimizzare la quantità di filo elettrico per collegare fra loro i diversi componenti.

(cont.)

- Ogni grafo non orientato (non necessariamente connesso) ha un **minimum spanning forest**, che è l' unione degli MST delle sue componenti connesse.

Un altro esempio

- Supponiamo che una compagnia telefonica voglia cablare in fibra ottica una città.
- Se è costretta a tracciare i cavi lungo cammini predeterminati, allora potremo rappresentare la città mediante un grafo, in cui ogni casa è un vertice.
- Alcuni cammini potrebbero essere più costosi di altri in quanto potrebbero richiedere più cavo o potrebbero richiedere scavi più profondi.
- Uno **spanning tree** per quel grafo sarà un insieme di quei cammini che connettono ogni casa della città.

(cont.)

- Tra i differenti spanning trees possibili, MST sarà quello *minimum spanning tree* con il costo (peso) complessivo più basso.

MST: Proprietà

- **Possible multiplicity (Possono esistere diversi MST)**
 - in particolare se tutti i pesi degli archi sono gli stessi ciascun spanning tree è un MST.
- **Uniqueness**
 - Se ciascun arco ha un peso differente allora esiste uno ed uno MST (vero in molte situazioni realistiche)
- **Minimum-cost subgraph**
 - Se tutti i pesi sono non negativi allora un MST è il minimum-cost subgraph che connette tutti i vertici.

(cont.)

- **Cycle property**

- Per ogni ciclo C nel grafo, se il peso di un arco e di C è più grande dei pesi degli altri archi di C , allora e non può appartenere ad un MST.

- **Cut property**

- Per ogni cut C nel grafo, se il peso di un arco e di C è più piccolo dei pesi degli altri archi di C , allora e appartiene a tutti gli MST del grafo.

MST: Storia

- Il primo algoritmo per trovare un MST fu sviluppato dall'ingegnere boemo **Otakar Borůvka** nel 1926 allo scopo di trovare una efficiente copertura elettrica della Moravia. L' **algoritmo di Boruvka** era greedy.
- Gli algoritmi comunemente usati sono : gli algoritmi di **Prim** e di **Kruskal**.

(cont.)

- Il più veloce algoritmo per MST è stato sviluppato da **Bernard Chazelle**, ed è basato su quello di Borůvka. La sua complessità è $O(E \alpha(E, V))$, dove E è il numero degli archi, V è il numero dei vertici e α è l'inversa della funzione di Ackermann. La funzione α cresce molto lentamente, tanto da poter essere considerata per scopi pratici non superiore a 4; pertanto l'**algoritmo di Chazelle** ha **complessità computazionale prossima a $O(E)$ time**.

(cont)

- Qual è il più veloce algoritmo per questo problema ?
- Questa domanda è uno dei problemi (più vecchi) aperti dell' informatica.
- Esiste un limite lineare, in quanto si devono esaminare almeno tutti i pesi.
- Se i pesi sono interi con una bit length limitata allora sono disponibili algoritmi la cui complessità è $O(E)$. L' **esistenza di un algoritmo con complessità lineare per pesi generali** è ancora un problema aperto.

Definizione del problema MST

Input:

- $G = (V, E)$ un grafo non orientato e connesso
- $w: E \rightarrow \mathbb{R}$ una funzione di peso (costo di connessione)
- se $(u, v) \in E$, allora $w(u, v)$ è il peso dell'arco (v, u)
- se $(u, v) \notin E$, allora $w(u, v) = \infty$.
- Poiché G non è orientato, $w(u, v) = w(v, u)$

Albero di Copertura (spanning tree)

- Dato un grafo $G = (V, E)$ non orientato e connesso, un albero di copertura di G è un sottografo $T = (V, E_T)$ tale che:
 - T è un albero
 - $E_T \subseteq E$
 - T contiene tutti i vertici di G

Minimum Spanning Tree (MST)

- Il Minimum Spanning Tree (Albero di Copertura minimo) è quell' albero di copertura per cui il suo peso totale: $\sum_{(u,v) \in T} w(u, v)$ è **minimo**.
- Il minimum spanning tree può non essere unico.

Algoritmo greedy generico

- Vediamo un algoritmo di tipo greedy *generico* e due “istanze” di questo algoritmo: **Kruskal** e **Prim**.
- L'idea è di *accrescere* un sottoinsieme A di archi in modo tale che venga rispettato il seguente invariante di ciclo:
 - A è un sottoinsieme di qualche albero di connessione minimo

Arco Sicuro

- Un arco (u, v) è detto **sicuro** per A se **$A \cup \{(u, v)\}$ è ancora un sottoinsieme di qualche albero di connessione minimo.**

Algoritmo generico

Generic-MST(G, w)

1. $A \leftarrow \emptyset$
2. **while** *A non forma un albero di copertura*
3. trova un arco sicuro (u, v)
4. $A \leftarrow A \cup \{(u, v)\}$
5. **return** A

Inizializzazione

- Dopo la riga 1, l'insieme A soddisfa banalmente l'invariante di ciclo.

Conservazione

- Il ciclo nelle righe 2-4 conserva l'invariante perchè aggiunge solo archi sicuri.

Conclusione

- Tutti gli archi aggiunti ad A si trovano in un albero di connessione minimo, quindi l'insieme A restituito nella riga 5 deve essere un albero di connessione minimo.

Come trovare un arco sicuro ?

- Un arco sicuro deve esistere, perché quando viene eseguita la riga 3, l' invariante impone che ci sia un albero di connessione T tale che $A \subseteq T$.
- All' interno del corpo while, A deve essere un sottinsieme proprio di T , quindi deve esistere un arco $(u, v) \in T$ tale che $(u, v) \notin A$ e (u, v) è un arco sicuro per A .

Definizioni

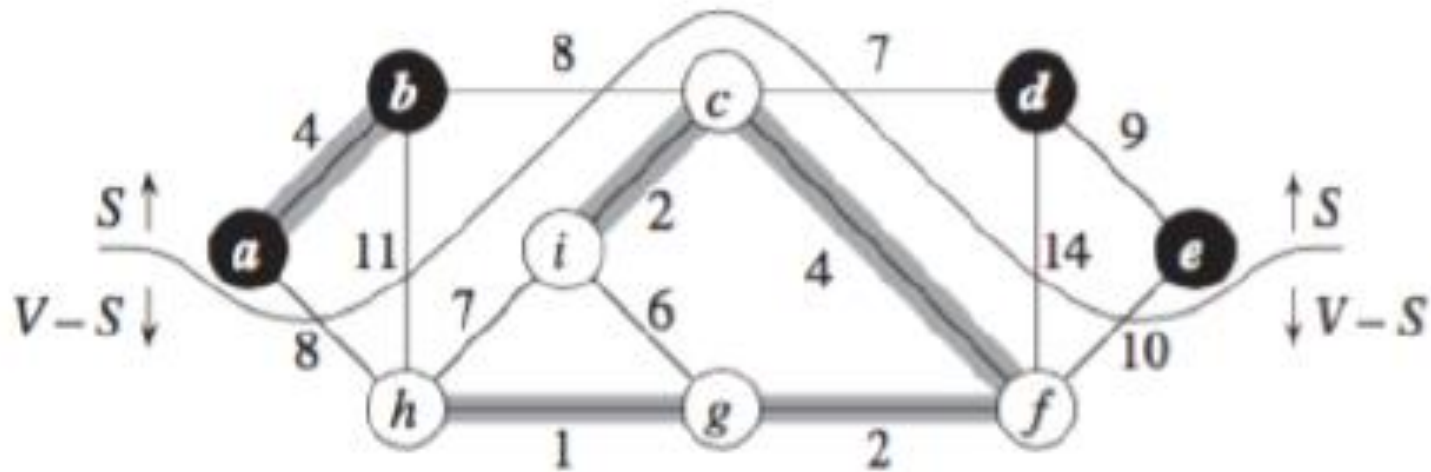
Per caratterizzare gli archi sicuri dobbiamo introdurre alcune definizioni:

- Un **taglio** (*cut*) $(S, V - S)$ di un grafo non orientato $G = (V, E)$ è una partizione di V in due sottoinsiemi disgiunti
- Un arco (u, v) **attraversa** il taglio se $u \in S$ e $v \in V - S$
- Un taglio **rispetta** un insieme di archi A **se nessun arco di A attraversa il taglio.**

(cont.)

- Un arco che attraversa un taglio è *leggero* nel taglio **se il suo peso è minimo fra i pesi degli archi che attraversano un taglio**

Esempio



Teorema: Archi sicuri

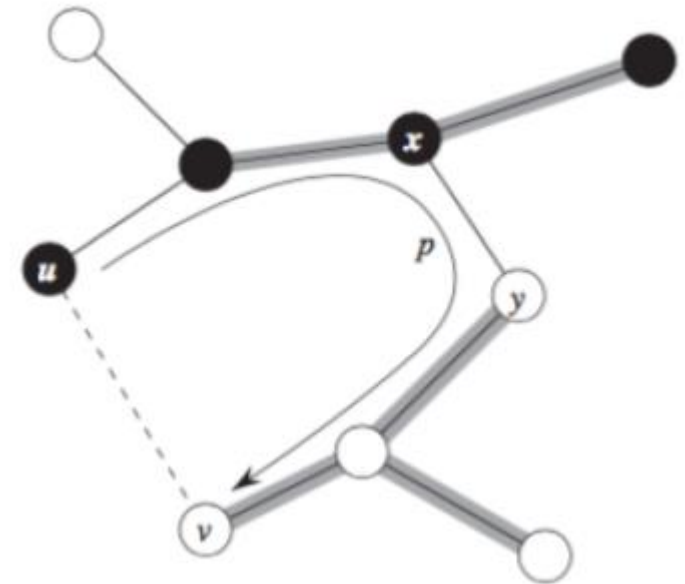
- Sia $G = (V, E)$ un grafo non orientato e connesso
- Sia w una funzione peso a valori reali definita su E .
- Sia $A \in E$ contenuto in un qualche albero di copertura minimo per G .
- Sia $(S, V - S)$ un taglio qualsiasi di G che rispetta A .
- Sia (u, v) un arco leggero che attraversa il taglio $(S, V - S)$ ***allora l'arco (u, v) è sicuro per A***

Dimostrazione

- Sia T un albero di connessione minimo che contiene A e supponiamo che T non contenga l'arco leggero (u, v) , perchè se lo contenesse il teorema sarebbe dimostrato.
- Costruiremo un altro albero di connessione minimo T' che include $A \cup (u, v)$ utilizzando una tecnica cut and paste dimostrando così che (u, v) è un arco sicuro per A

(cont.)

- L' arco (u, v) forma un ciclo con gli archi nel cammino semplice p che va da u a v in T . Poiché u e v si trovano sui lati opposti del taglio $(S, V - S)$, c' è almeno un altro arco in T che appartiene al cammino semplice p che attraversa il taglio.



(cont.)

- Sia (x, y) uno di questi archi. L' arco (x, y) non appartiene ad A , perché il taglio rispetta A .
- Poiché (x, y) si trova nel cammino semplice unico da u a v in T eliminando (x, y) , l' albero T si spezza in due componenti.
- Aggiungendo (u, v) le due componenti si ricongiungono per fornire un nuovo albero di copertura $T' = T - \{(x, y)\} \cup (u, v)$.

(cont.)

- Poiché (u, v) è un arco è un arco leggero che attraversa $(S, V - S)$ e anche l' arco (x, y) che attraversa lo stesso taglio, allora si ha $w(u, v) \leq w(x, y)$. Quindi si ha:
$$w(T') = w(T) - w(x, y) + w(u, v) \leq w(T)$$
- Ma poiché T è un albero di connessione minimo, quindi $w(T) \geq w(T')$; di conseguenza anche T' deve essere un albero di connessione minimo.

Corollario: Archi sicuri

- Sia $G = (V, E)$ un grafo non orientato e connesso
- Sia w una funzione peso a valori reali definita su E .
- Sia $A \in E$ contenuto in un qualche albero di copertura minimo per G .
- Sia C una componente connessa (un albero) nella foresta $G_A = (V, A)$
- Sia (u, v) un arco leggero che connette C a qualche altra componente in G_A ***allora l'arco (u, v) è sicuro per A***

Dimostrazione

- Il taglio $(V_C, V - V_C)$ rispetta A e (u, v) è un arco leggero per questo taglio. Quindi (u, v) è sicuro per A .

Costruzione di un albero di copertura minima

- Algoritmo di **Kruskal** (1956)
- Algoritmo di **Prim** (1957)
 - L' algoritmo fu scoperto nel 1930 dal matematico **Vojtěch Jarník** e più tardi indipendentemente da **Robert C. Prim** nel 1957 e riscoperto da **Edsger Dijkstra** nel 1959. Perciò è talvolta chiamato **DJP algorithm**, o **Jarník algorithm**, o **Prim-Jarník algorithm**.

Algoritmo di Kruskal: idea

- Ingrandire sottoinsiemi disgiunti di un albero di copertura minimo connettendoli fra di loro fino ad avere l'albero complessivo
- Si individua un arco sicuro scegliendo un arco (u, v) di peso minimo tra tutti gli archi che connettono due distinti alberi (componenti connesse) della foresta
- L'algoritmo è greedy perché ad ogni passo si aggiunge alla foresta un arco con il peso minore

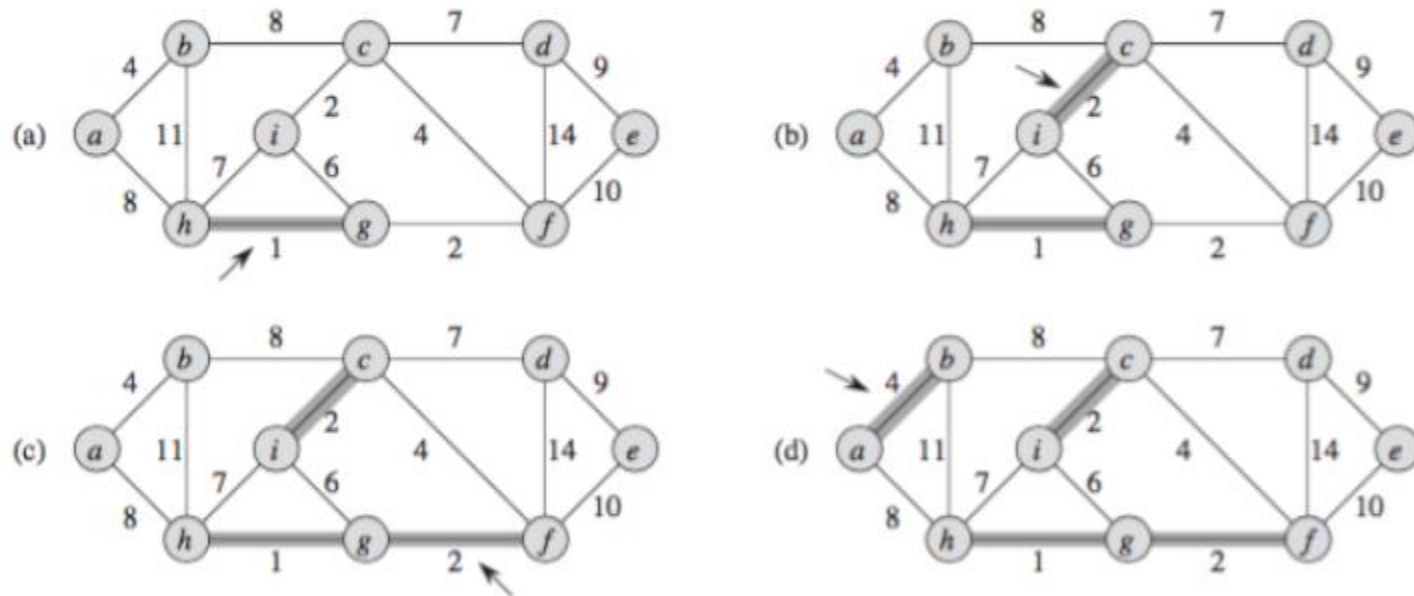
Algoritmo di Kruskal

1. $A \leftarrow \emptyset$
2. **for** ogni vertice $v \in V[G]$
3. do MAKE-SET(V)
4. Sort(E)
5. // Ordina gli archi di E per peso w non decrescente
6. **for** ogni arco $(u, v) \in E[G]$
7. **if** FIND-SET(u) $\neq$ FIND-SET(v)
8. $A = A \cup \{(u, v)\}$
9. Union(u, v)
10. **return** A

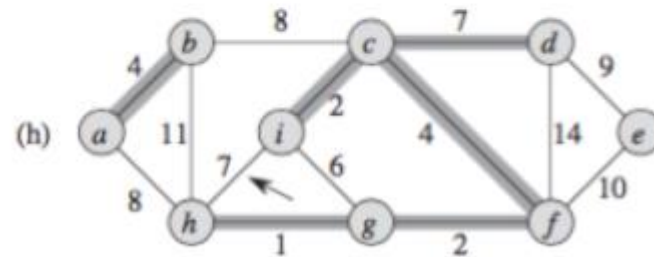
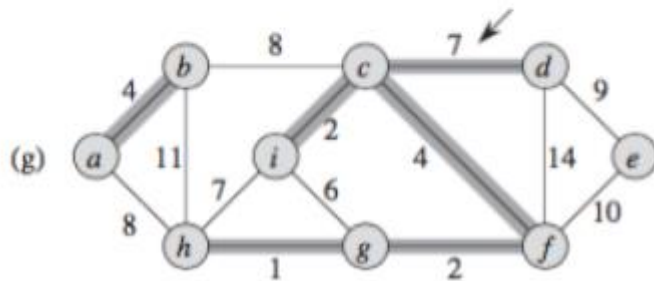
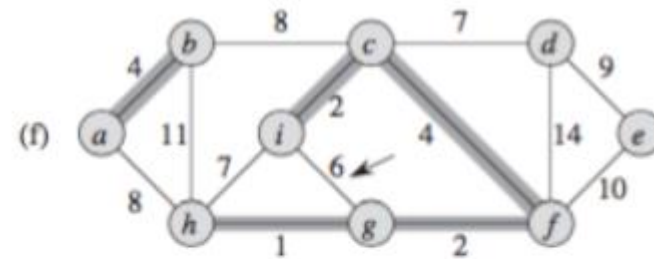
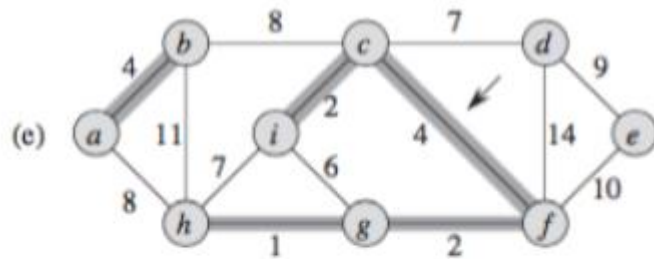
Note

- Linee 1-4: A viene inizializzato con l'insieme vuoto; MAKE-SET crea $|V|$ alberi, uno per ogni vertice; gli archi di E vengono ordinati in ordine di peso non decrescente
- Linee 6-9: $FIND_SET(u)$ restituisce un rappresentante dell'insieme che contiene u ; perciò si determina se due vertici u e v appartengono allo stesso albero. Se i due vertici appartengono ad alberi diversi, l'arco (u, v) viene aggiunto ad A ed i due alberi vengono fusi, mediante la UNION, in un unico albero.
- Linea 10: Viene restituito A

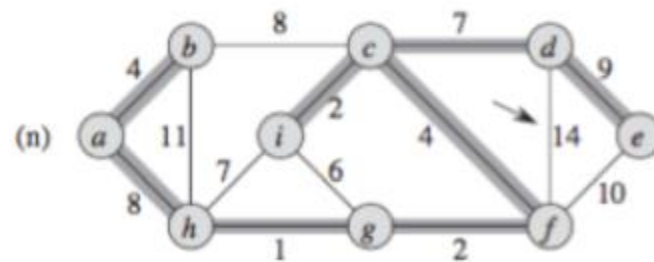
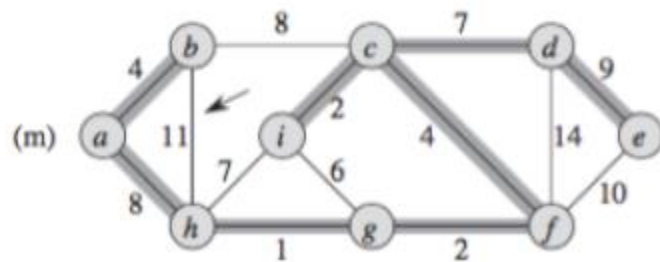
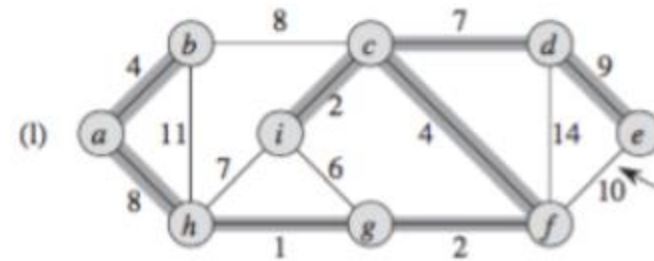
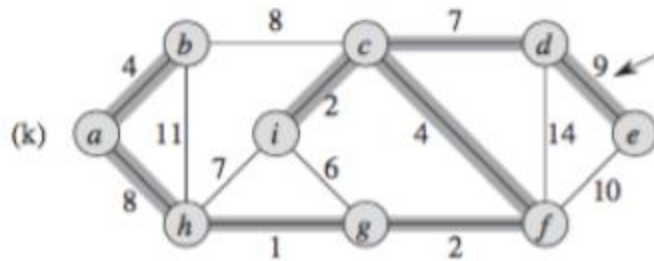
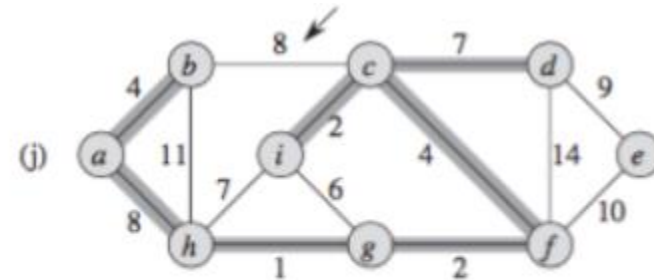
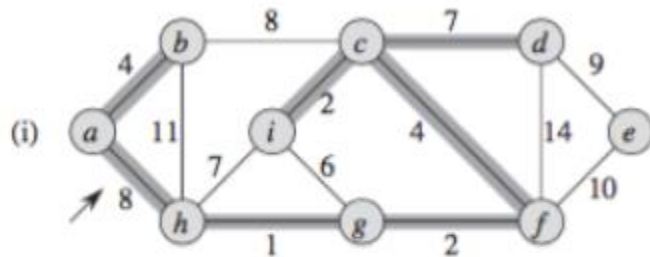
Esempio



(cont.)



(cont.)



Complessità dell' algoritmo di Kruskal

- L' inizializzazione richiede tempo $O(V)$
- Il tempo necessario per ordinare gli archi è
($|E| < |V| * |V|$) $O(E \log E) = O(E \log V^2) =$
 $O(2E \log V) = O(E \log V)$
- Le operazioni sulla foresta di insiemi disgiunti è
 $O(E \alpha(E, V))$ circa $O(E)$
- Quindi il tempo di esecuzione totale è pari a
 $O(V + E \log V + E \alpha(E, V)) = O(E \log V)$.